# Name :  Vivek Shashikant Chaudhari

# MediPulse

Healthcare Patient & Hospital Operations Intelligence

Turning Healthcare Data into Actionable Insights

## Healthcare Patient & Hospital Operations Intelligence

**Tagline:** Turning Healthcare Data into Actionable Insights

### 1. Project Overview

MediPulse is an end-to-end healthcare analytics project designed to analyze patient admissions, hospital operations, medical conditions, billing, insurance providers, medications and clinical test results.

The project transforms raw healthcare data into a structured, cleaned and analysis-ready dataset. Python is used for data exploration, quality checks, cleaning and feature engineering. MySQL is used to store the data in a star schema and perform business-focused SQL analysis. Power BI is used to develop interactive dashboards that communicate important findings through KPIs, charts and filters.

### 2. Business Problem

Healthcare organizations need to understand patient admission patterns, hospital performance, billing trends, length of stay and clinical outcomes. Without organized analysis, identifying patterns and areas that may require further investigation can be difficult.

MediPulse addresses these analytical needs by organizing healthcare records and presenting measurable insights that can support operational monitoring and financial review.

### 3. Dataset Overview

The project uses a CSV file named `healthcare_dataset.csv`.

| Dataset property | Details |
|---|---|
| Original records | 55,500 |
| Original columns | 15 |
| Data format | CSV |
| Analysis tools | Python, MySQL and Power BI |
| Admission date range | 8 May 2019 – 7 May 2024 |
| Main data categories | Patient demographics, admissions, hospitals, billing and clinical information |

## 4. Original Dataset Columns

1. `Name` — Patient name recorded in the dataset.
2. `Age` — Patient age.
3. `Gender` — Recorded gender.
4. `Blood Type` — Recorded blood group.
5. `Medical Condition` — Medical condition associated with the admission.
6. `Date of Admission` — Date the patient was admitted.
7. `Doctor` — Doctor name recorded for the admission.
8. `Hospital` — Hospital name.
9. `Insurance Provider` — Insurance provider associated with the record.
10. `Billing Amount` — Recorded billing amount.
11. `Room Number` — Assigned room number.
12. `Admission Type` — Emergency, Urgent or Elective.
13. `Discharge Date` — Date the patient was discharged.
14. `Medication` — Medication recorded in the dataset.
15. `Test Results` — Normal, Abnormal or Inconclusive.

## 5. Initial Data Understanding

The initial dataset contained 55,500 records and 15 original columns. The fields covered demographic, operational, financial and clinical information, making it possible to investigate several healthcare business questions.

Initial inspection also identified data-quality issues that required further investigation during the Python stage:

- 534 exact duplicate rows.
- 108 records with negative billing amounts.
- No missing values were reported in the initial inspection.

- Admission and discharge dates required validation.
- Length of stay was not an original column and needed to be derived from the admission and discharge dates.

The negative billing values were treated as records requiring review, not automatically as confirmed billing errors or fraud.

### 6. Important Data Limitation

The original dataset does not contain a dedicated, verified patient ID. Consequently, repeated patient names cannot by themselves establish that records belong to the same real person. Patient-level analysis must be interpreted according to the constructed analytical key and its limitations.

### 7. Initial Business Questions

The dataset was selected to investigate questions such as:

- How many admissions are recorded, and how does demand change over time?
- Which medical conditions account for the most admissions?
- How do admission types differ across the dataset?
- What are the average billing amount and length of stay?
- How is recorded revenue distributed across hospitals and insurance providers?
- What proportion of admissions have abnormal test results?
- Which records or patterns may require data-quality, operational or financial review?

**Outcome of Step 1:** The dataset structure, business context, original fields, initial data-quality concerns and analytical questions were documented before beginning Python-based exploration and cleaning.

# Step 2: Python — Exploratory Data Analysis & Data Cleaning

## 2.1 Objective

The objective of the Python stage was to understand the healthcare dataset, identify data-quality issues, explore important patterns, create additional analytical features and prepare a reliable dataset for MySQL-based analysis.

**Tool used:** PyCharm

**Programming language:** Python

**Libraries:** Pandas, NumPy and Matplotlib

**Input file:** `healthcare_dataset.csv`

**Output files:**

- `medipulse_feature_engineered.csv`
- `medipulse_cleaned.csv`
- `medipulse_negative_billing_review.csv`

## 2.2 Loading and Inspecting the Dataset

The project started by loading the CSV file into a Pandas DataFrame.

Initial inspection was performed using functions such as `head()`, `shape`, `columns`, `info()`, `describe()` and data-type checks.

The initial dataset contained **55,500 rows and 15 columns**.

This stage helped identify the structure of the dataset, understand the data types and review the numerical and categorical fields before performing detailed analysis.

## 2.3 Exploratory Data Analysis (EDA)

Exploratory Data Analysis was performed to understand the distribution of patient admissions, medical conditions, billing amounts, hospital records and clinical information.

### A. Categorical Analysis

Frequency counts and percentage distributions were examined for important categorical fields, including:

- Gender
- Blood Type
- Medical Condition
- Insurance Provider
- Admission Type
- Medication

- Test Results

The dataset contained six medical conditions: Arthritis, Diabetes, Hypertension, Obesity, Cancer and Asthma.

Admission types were classified as Emergency, Urgent and Elective. Test results were categorized as Normal, Abnormal and Inconclusive.

The distributions were relatively balanced across several categories. This was documented as a characteristic of the dataset rather than evidence of real-world healthcare demand.

## B. Cross-Category Analysis

Cross-tabulations were used to examine relationships between categorical variables.

Examples included:

- Medical Condition versus Gender
- Medical Condition versus Admission Type
- Medical Condition versus Test Results
- Insurance Provider versus Admission Type

These comparisons helped assess how records were distributed across different combinations of patient, admission and clinical categories.

## C. Numerical Analysis

Descriptive statistics were calculated for Age, Billing Amount, Room Number and Length of Stay.

| Metric | Initial dataset result |
|---|---|
| Average age | 51.54 years |
| Median age | 52 years |
| Average billing amount | 25,539.32 |
| Median billing amount | 25,538.07 |
| Minimum age | 13 years |
| Maximum age | 89 years |

The billing figures are in the dataset's recorded monetary units; the currency was not independently established from the source data.

### D. Business Analysis

Numerical measures were compared across categories to investigate potential differences in billing and length of stay.

Examples included:

- Average billing by medical condition
- Average length of stay by admission type
- Billing by insurance provider
- Length of stay by medical condition
- Average billing by age group and medication

These analyses helped establish the baseline for later SQL queries and Power BI visualizations.

## 2.4 Data Quality Assessment

A detailed quality assessment was conducted before cleaning.

| Quality check | Initial finding |
|---|---|
| Total records | 55,500 |
| Exact duplicate rows | 534 |
| Negative billing records | 108 |
| Missing values | 0 |
| Invalid age records under the selected rule | 0 |
| Invalid admission/discharge date order | 0 |
| Negative length of stay | 0 |
| Length of stay above 30 days | 0 |
| Invalid room numbers under the selected rule | 0 |

The age validation rule checked for values below zero or above 120. Date validation checked whether discharge occurred before admission. Length of stay was derived from the difference between discharge and admission dates.

The billing analysis identified 108 records with negative billing amounts. The minimum recorded billing amount was approximately -2,008.49.

These records were flagged for review because negative billing could represent an adjustment, refund, or data-quality issue. The dataset alone did not establish the underlying cause.

## 2.5 Duplicate and Patient-Name Analysis

The initial dataset contained **534 exact duplicate rows**.

Repeated names were also examined. However, a repeated name was not automatically treated as a duplicate patient because the dataset did not contain a verified patient identifier.

Exact row duplication and repeated patient names were therefore treated as two different checks.

## 2.6 Outlier Analysis

The Interquartile Range (IQR) method was used to assess potential numerical outliers in Billing Amount, Age and Length of Stay.

The method calculates:

- IQR = Q3 − Q1
- Lower boundary = Q1 − 1.5 × IQR
- Upper boundary = Q3 + 1.5 × IQR

The selected IQR rules did not identify any outliers in these three numerical fields. However, the negative billing records still required separate investigation because business validity is not determined by the IQR method alone.

## 2.7 Feature Engineering

Additional features were created to make the data more useful for analysis and reporting.

### Date and Time Features

- Admission Year
- Admission Month
- Admission Month Name
- Admission Quarter
- Admission Day
- Admission Weekday
- Weekend Admission
- Admission Season

### Patient and Stay Features

- Age Group
- Patient Age Band
- Length of Stay
- LOS Category
- Long Stay Flag
- Stay Severity
- Patient Stay Segment

### Financial and Operational Features

- Billing Category
- High Billing Flag
- Emergency Flag
- Abnormal Test Flag
- Weekend Flag
- Revenue Per Stay Day

These features supported time-based analysis, patient segmentation, operational monitoring and financial analysis.

The feature-engineered dataset contained **55,500 records and 36 columns** and was exported as `medipulse_feature_engineered.csv`.

## 2.8 Data Cleaning

After EDA and feature engineering, the dataset was cleaned for downstream analysis.

The cleaning workflow included:

1. Removing exact duplicate rows.
2. Saving negative billing records separately for review.
3. Excluding records with negative billing amounts from the analytical dataset.
4. Standardizing categorical text and formatting.
5. Validating numerical and date fields.
6. Recalculating Length of Stay.
7. Running final data-quality checks.

The cleaning decisions were applied to create a consistent dataset for the SQL data warehouse.

## 2.9 Final Validation

| Metric | Before cleaning | After cleaning |
|---|---|---|
| Records | 55,500 | 54,860 |
| Exact duplicate rows | 534 | 0 |
| Negative billing records | 108 | 0 |
| Missing values | 0 | 0 |

The final cleaned dataset contained **54,860 records and 36 columns**.

The record-count reduction was 640. Of these, 534 records were removed through exact duplicate elimination and 106 remaining negative-billing records were excluded. Two negative-billing records were already removed during duplicate elimination.

The negative-billing records were preserved separately in `medipulse_negative_billing_review.csv` for potential review.

The final cleaned file was exported as `medipulse_cleaned.csv`.

## 2.10 Outcome of the Python Stage

The Python stage transformed the raw healthcare dataset into a cleaned and feature-engineered analytical dataset.

The resulting data was ready to be loaded into MySQL for dimensional modelling, relationship creation, KPI calculations and business-focused SQL analysis.

**Key deliverables:**

- Initial EDA and descriptive statistics
- Categorical and numerical analysis
- Data-quality assessment
- Feature-engineered dataset
- Separate negative-billing review file
- Final cleaned dataset containing 54,860 records

# Step 3: MySQL — Data Warehouse Design & SQL Analysis

## 3.1 Objective

The objective of the MySQL stage was to store the cleaned healthcare dataset in a structured relational database, design a star schema, establish relationships between fact and dimension tables, and perform business-focused SQL analysis.

**Tool used:** MySQL Workbench

**Database:** `medipulse`

**Input file:** `medipulse_cleaned.csv`

**Input records:** 54,860

## 3.2 Database Creation

A database named `medipulse` was created to store the cleaned data and analytical tables.

```
CREATE DATABASE medipulse;

USE medipulse;
```

This database served as the central data warehouse for the SQL analysis and subsequent Power BI integration.

## 3.3 Data Import into the Staging Table

The cleaned CSV file was imported into a staging table named `staging_admissions`.

The staging table acted as the initial landing area for the cleaned records before they were distributed into the dimensional model.

The data was loaded using MySQL's `LOAD DATA LOCAL INFILE` command after configuring the client connection to permit local file loading.

The imported record count was verified using:

```
SELECT COUNT(*) AS total_records
FROM staging_admissions;
```

**Result:** 54,860 records.

## 3.4 Star Schema Design

A star schema was designed to organize the healthcare data into a central fact table connected to descriptive dimension tables.

The central fact table stores admission-level measures and foreign keys. The dimension tables provide descriptive information for filtering, grouping and reporting.

### Fact Table

#### `fact_admission`

The fact table contains one row per analytical admission record.

Key columns include:

- `admission_key` — Primary key for the fact record.
- `patient_key` — Reference to the patient dimension.
- `hospital_key` — Reference to the hospital dimension.
- `condition_key` — Reference to the medical-condition dimension.
- `insurance_key` — Reference to the insurance dimension.
- `medication_key` — Reference to the medication dimension.
- `admission_type_key` — Reference to the admission-type dimension.
- `test_result_key` — Reference to the test-result dimension.
- `date_key` — Reference to the admission-date dimension.
- `billing_amount` — Recorded billing amount.
- `room_number` — Recorded room number.
- `length_of_stay` — Calculated length of stay in days.
- `revenue_per_stay_day` — Billing amount divided by length of stay.

### Dimension Tables

| Dimension table | Purpose | Row count |
|---|---|---|
| `dim_patient` | Patient profile attributes | 54,838 |
| `dim_hospital` | Hospital names and identifiers | 39,815 |
| `dim_medical_condition` | Medical-condition categories | 6 |
| `dim_insurance` | Insurance providers | 5 |

| | | |
|---|---|---|
| dim_medication | Medication categories | 5 |
| dim_admission_type | Admission categories | 3 |
| dim_test_result | Clinical test-result categories | 3 |
| dim_date | Date attributes for time-based analysis | 1,827 |
| fact_admission | Admission-level measures and keys | 54,860 |

The patient dimension uses available profile attributes because the source dataset did not contain a verified patient ID. Therefore, its 54,838 rows represent constructed analytical profiles, not a confirmed count of unique real-world patients.

## 3.5 Relationships and Referential Integrity

The fact table was connected to eight dimension tables using foreign-key relationships.

The relationships were configured as one-to-many from each dimension to the fact table.

The eight relationships were:

1. dim_patient → fact_admission
2. dim_hospital → fact_admission
3. dim_medical_condition → fact_admission
4. dim_insurance → fact_admission
5. dim_medication → fact_admission
6. dim_admission_type → fact_admission
7. dim_test_result → fact_admission
8. dim_date → fact_admission

Foreign-key constraints were added to enforce valid references. Indexes were also created on relevant fact-table foreign keys and dimension business-name columns to support common analytical queries.

The fact-table loading process was validated using joins, confirming that all 54,860 source records matched the required dimension records before insertion.

## 3.6 KPI Calculations Using SQL

SQL aggregation functions were used to calculate the baseline KPIs for the project.

| KPI | Result |
|---|---|
| Total Admissions | 54,860 |

| | |
|---|---|
| Total Patients* | 54,838 |
| Total Hospitals* | 39,815 |
| Total Revenue | 1,404,121,599.97 |
| Average Billing | 25,594.63 |
| Average Length of Stay | 15.50 days |
| Maximum Length of Stay | 30 days |

*The patient and hospital counts reflect distinct keys in the constructed dimensions. The patient count should not be interpreted as a verified count of unique individuals.*

Example SQL query:

```
SELECT
    COUNT(*) AS total_admissions,
    ROUND(SUM(billing_amount), 2) AS total_revenue,
    ROUND(AVG(billing_amount), 2) AS average_billing,
    ROUND(AVG(length_of_stay), 2) AS average_length_of_stay,
    MAX(length_of_stay) AS maximum_length_of_stay
FROM fact_admission;
```

This query provides a high-level summary of admission volume, recorded revenue and length-of-stay measures.

## 3.7 SQL Business Analysis

Multiple SQL analyses were developed to answer operational, clinical and financial questions.

### A. Hospital Performance Analysis

Hospital-level queries calculated admission volume, total revenue, average billing and average length of stay.

Ranking queries were used to identify hospitals with high revenue and high admission volumes.

### B. Medical Condition Analysis

Queries compared admission counts, billing measures and length of stay across the six medical conditions.

Revenue contribution percentages were also calculated to understand each condition's share of total recorded revenue.

## C. Admission Type Analysis

Emergency, Urgent and Elective admissions were compared by volume, revenue and length of stay.

This supported the analysis of admission patterns across the recorded categories.

## D. Insurance Provider Analysis

Queries compared admission volume and billing across insurance providers. Insurance provider and admission type combinations were also examined.

## E. Medication and Test Result Analysis

Medication-level and test-result-level queries compared admission counts, billing amounts and length of stay.

These analyses helped explore differences between the recorded treatment and clinical-result categories.

## F. Age Group and Gender Analysis

CASE expressions and grouping logic were used to classify age bands and compare admission patterns across age, gender and medical condition.

## G. Time-Based Analysis

Year-wise and month-wise analyses were performed to examine changes in admissions and revenue.

Common table expressions and the `LAG()` window function were used for year-over-year revenue comparisons. Running totals were calculated with windowed `SUM()` expressions.

## H. Ranking and Top-N Analysis

Window functions including `RANK()`, `DENSE_RANK()` and `ROW_NUMBER()` were used to rank hospitals by revenue or admission volume and select top-performing hospitals.

## I. Hospital Capacity and Long-Stay Analysis

Queries measured the number and percentage of admissions with a length of stay greater than 20 days.

These results were treated as indicators for operational review, not proof of bed shortages, because actual bed capacity and occupancy data were unavailable.

### J. Financial Review and High-Billing Analysis

Hospital billing measures were compared with network-level averages to identify records and hospitals that could merit further financial review.

Percentile ranking was used to identify admissions in the top 5% of billing amounts, including condition-specific percentile analysis.

These were screening indicators only. High billing or low average billing does not establish fraud, billing errors or revenue leakage.

### K. Monthly Demand Analysis

Monthly hospital admission volumes were compared with each hospital's average monthly volume. Emergency-admission percentages were included to identify periods that might warrant resource-planning review.

Because some hospitals had low record counts, the results were interpreted cautiously rather than as definitive evidence of demand surges.

## 3.8 SQL Concepts Applied

The project used a range of SQL concepts:

- Database and table creation
- Data loading and validation
- Primary keys and foreign keys
- Joins
- Aggregate functions such as `COUNT()`, `SUM()` and `AVG()`
- `GROUP BY` and `HAVING`
- `CASE` expressions
- Common Table Expressions (CTEs)
- Window functions
- `LAG()`, `SUM() OVER()`, `RANK()`, `DENSE_RANK()` and `ROW_NUMBER()`
- Percentile-based ranking
- Indexes and relationship validation

These techniques supported reusable queries for business-focused analysis rather than relying only on basic record counts.

## 3.9 SQL Stage Outcome

The MySQL stage transformed the cleaned dataset into a structured analytical database.

**Key deliverables:**

- `medipulse` database
- `staging_admissions` staging table
- Eight dimension tables
- One central fact table
- Eight foreign-key relationships
- Verified loading of 54,860 fact records
- KPI calculations and business-analysis queries
- Hospital, clinical, operational and financial analysis results

The completed database was then connected to Power BI Desktop to build interactive dashboards using the fact and dimension tables.

## 3.10 Limitations

The analysis was constrained by the fields available in the source dataset. It did not include verified patient identifiers, hospital bed capacity, staffing levels, insurance claim status or clinical treatment outcomes beyond the recorded test-result categories.

Consequently, findings were used for descriptive analysis and review prioritization rather than causal conclusions, clinical recommendations or fraud determinations.

## 3.11 Connecting MySQL with Power BI Desktop

### Objective

The objective of this step was to connect the MySQL database to Power BI Desktop so that the structured healthcare data could be used to build interactive dashboards and reports.

### Tools and Requirements

- MySQL Server — Stores the MediPulse database.

- MySQL Workbench — Used to create tables, run SQL queries and validate the data.
- Power BI Desktop — Used to import data, create DAX measures and develop dashboards.
- MySQL Connector/NET — Installed to enable the Power BI MySQL connector to communicate with MySQL.

## Step 1: Install MySQL Connector/NET

The MySQL Connector/NET driver was installed because Power BI required the appropriate MySQL client provider to establish the database connection.

During setup, a connector compatibility issue occurred. The installed provider registrations were inspected and corrected so that Power BI could recognize the MySQL connector successfully.

## Step 2: Open the MySQL Connector in Power BI

Power BI Desktop was opened, and the following navigation was used:

**Home → Get Data → More → Database → MySQL database → Connect**

The MySQL database connector was selected from the available data sources.

## Step 3: Enter Connection Details

The following connection details were entered:

| Setting | Value |
|---|---|
| Server | `localhost:3306` |
| Database | `medipulse` |
| Database system | MySQL |
| Data selection method | Navigator / Power Query |

The server address `localhost:3306` referred to the MySQL Server running on the same computer, using the default MySQL port.

## Step 4: Select the Required Tables

After the connection was established, Power BI displayed the tables available in the `medipulse` database.

The following nine tables were selected:

1. dim_admission_type
2. dim_date
3. dim_hospital
4. dim_insurance
5. dim_medical_condition
6. dim_medication
7. dim_patient
8. dim_test_result
9. fact_admission

The staging_admissions table was intentionally excluded because it was an intermediate staging table and the dashboards were designed to use the final dimensional model.

## Step 5: Transform Data

Instead of immediately loading the selected tables, **Transform Data** was chosen to open Power Query Editor.

The selected tables were reviewed before loading. This provided an opportunity to inspect the imported data and perform any required transformations.

After reviewing the queries, **Close & Apply** was selected to load the tables into the Power BI data model.

## Step 6: Verify the Data Model

The Model view in Power BI was opened to verify the relationships between fact_admission and the eight dimension tables.

The model followed the same star-schema structure created in MySQL:

- Dimension tables were on the one side.
- fact_admission was on the many side.
- Each dimension was connected to the fact table using its corresponding key.

The relationships were checked to ensure that dimensions could filter the fact table and support the dashboard calculations.

## Step 7: Create DAX Measures

After loading the data, DAX measures were created for the main KPIs.

Example:

```
Total Admissions =
COUNTROWS('medipulse fact_admission')Total Revenue =
SUM('medipulse fact_admission'[billing_amount])Average Billing =
AVERAGE('medipulse fact_admission'[billing_amount])Average LOS =
AVERAGE('medipulse fact_admission'[length_of_stay])Total Patients =
DISTINCTCOUNT('medipulse fact_admission'[patient_key])Total
Hospitals =
DISTINCTCOUNT('medipulse fact_admission'[hospital_key])
```

These measures were used to display admission volume, recorded revenue, average billing, average length of stay and distinct dimension-key counts.

## Step 8: Validate the Results

The Power BI KPI results were compared with the SQL results to verify that the data had been imported correctly and the measures returned consistent values.

The principal reference values were:

- Total Admissions: 54,860
- Total Patients: 54,838 distinct patient keys
- Total Hospitals: 39,815 distinct hospital keys
- Total Revenue: 1,404,121,599.97
- Average Billing: 25,594.63
- Average Length of Stay: 15.50 days

## Outcome

The MySQL database was successfully connected to Power BI Desktop. Nine analytical tables were loaded, the star-schema relationships were verified, and DAX measures were created for dashboard reporting.

This connection enabled the development of five interactive MediPulse report pages covering overall healthcare performance, patient analytics, hospital operations, financial intelligence, and patient outcomes and clinical insights.

# Step 4.1: Power BI Dashboard — Page 1: Overall Healthcare Overview

## 1. Objective

The first Power BI report page was designed to provide a high-level overview of healthcare admissions, hospitals, patient profiles, recorded revenue and length of stay.

It serves as the main dashboard from which users can monitor key performance indicators (KPIs), compare healthcare categories and navigate to more detailed analytical pages.

## 2. Data Connection and Model

The dashboard uses data imported from the MySQL `medipulse` database into Power BI Desktop.

Nine tables were loaded: one fact table (`fact_admission`) and eight dimension tables. The star-schema relationships were verified in Model view before developing the visuals.

DAX measures were created to calculate key indicators, and slicers were added to allow users to filter the report.

## 3. KPI Cards

Six KPI cards were placed across the top section of the page.

| KPI | Displayed value | Purpose |
|---|---|---|
| Total Admissions | 54,860 | Shows the total number of analytical admission records. |
| Total Hospitals | 39,815 | Shows the distinct hospital keys represented in the model. |
| Total Patients | 54,838 | Shows the distinct patient keys represented in the model. |
| Total Revenue | 1.40bn | Shows the sum of recorded billing amounts. |

| | | |
|---|---|---|
| Average Billing | 25,594.63 | Shows the average billing amount per admission record. |
| Average LOS | 15.50 days | Shows the average length of stay in days. |

These cards provide a quick summary before the user examines individual charts.

**Important interpretation:** The patient and hospital figures are based on distinct analytical keys. The source data does not contain a verified patient ID, so the patient figure should not be interpreted as a confirmed count of unique real-world individuals.

# 4. Report Filters and Slicers

Three slicer controls are visible in the dashboard header.

### Year

Allows users to select an admission-year range, shown as 2019–2024.

### Medical Condition

Allows users to filter the report by a recorded medical condition, such as Arthritis, Diabetes, Hypertension, Obesity, Cancer or Asthma.

### Admission Type

Allows users to filter the report by Elective, Urgent or Emergency admission.

These slicers support interactive exploration of the report. Selecting a value can update connected visuals according to the configured visual interactions and model relationships.

# 5. Visualizations

### A. Insurance Provider Revenue Table

**Business question:** How is recorded revenue distributed across insurance providers?

The table displays insurance providers and their total revenue. It provides a direct comparison of the recorded billing contribution associated with each provider.

## B. Admissions by Admission Type — Donut Chart

**Business question:** How are admissions distributed across admission types?

The donut chart compares Elective, Urgent and Emergency admissions by volume and percentage.

The displayed distribution is approximately:

- Elective: 33.61%
- Urgent: 33.47%
- Emergency: 32.92%

This provides a quick view of the relative proportions of the three recorded admission categories.

## C. Average Length of Stay by Medical Condition — Bar Chart

**Business question:** How does average length of stay differ across medical conditions?

The horizontal bar chart compares average length of stay for the medical-condition categories. It helps users identify differences in average stay duration.

## D. Admissions by Medical Condition — Column Chart

**Business question:** Which medical conditions account for the most admission records?

The column chart compares admission counts across the six medical conditions. It makes category-level volume differences easier to assess.

The displayed counts are relatively similar across the conditions, reflecting the distribution in this dataset.

## E. Total Admissions by Year — Line Chart

**Business question:** How does admission volume change over time?

The line chart plots total admissions by admission year from 2019 to 2024.

It helps users identify changes in yearly volume. The chart shows a lower count in 2019, higher counts during 2020–2023, and a lower count in 2024.

**Interpretation caution:** The dataset covers only part of 2019 and part of 2024. The lower counts in those years may reflect incomplete calendar-year coverage, so they should not automatically be interpreted as a decline in healthcare demand.

### F. Top Hospitals by Admissions — Horizontal Bar Chart

**Business question:** Which hospitals have the highest admission volumes?

The chart displays the top five hospitals by admission count. It supports comparison of admission volumes among the selected hospitals.

The Top N filter was used to focus the visual on a manageable number of hospital names.

### G. Revenue by Medical Condition — Column Chart

**Business question:** How is recorded billing revenue distributed across medical conditions?

The column chart compares total billing revenue for each medical condition. It provides a financial comparison across the clinical categories recorded in the dataset.

The values are close to one another, so the chart is useful for comparing relative revenue contribution rather than suggesting large differences where the data does not show them.

## 6. Interactivity and Navigation

The report includes slicers for year, medical condition and admission type. These enable users to explore the available data without changing the underlying SQL queries.

A page-navigation panel on the left provides access to the five report pages:

1. Page 1 — Overall Healthcare Overview
2. Patient Analytics
3. Hospital & Operations
4. Financial Intelligence
5. Patient Outcomes & Clinical Insights

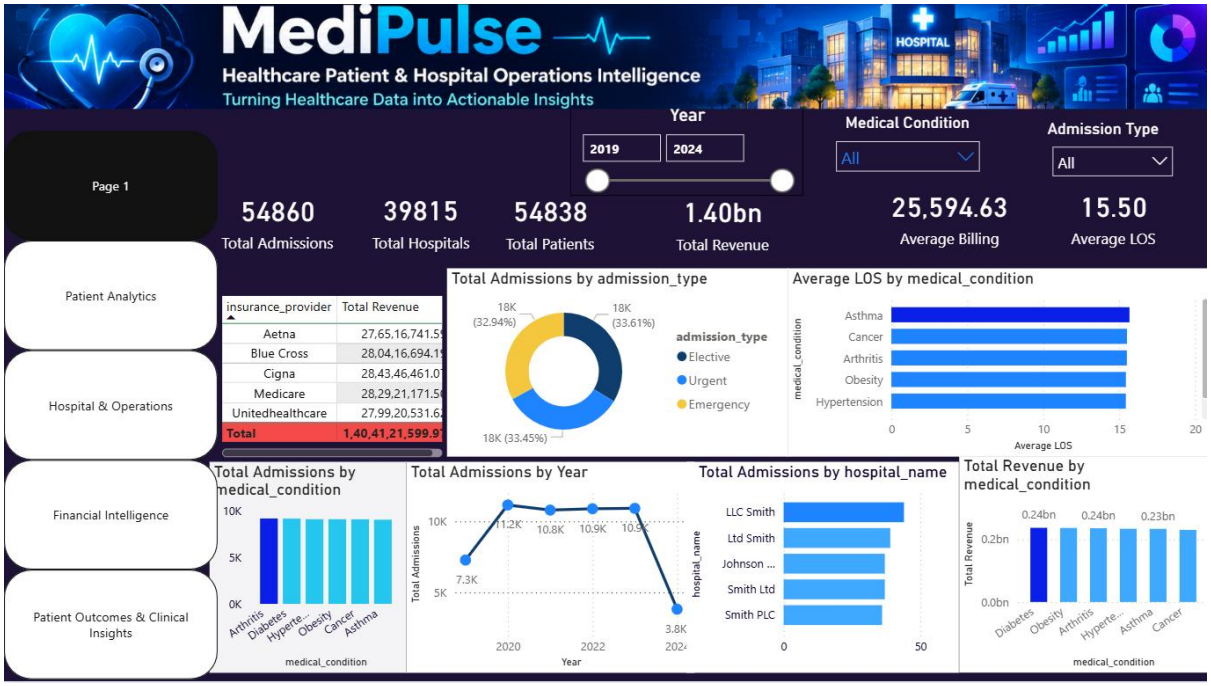
This makes Page 1 the main entry point for exploring the MediPulse report.

# 7. Business Value

The first dashboard page combines headline KPIs, category comparisons, a yearly trend and hospital-level ranking in a single view.

It helps users:

- Monitor overall admission volume and recorded revenue.
- Compare admission types and medical conditions.
- Review average billing and length of stay.
- Explore yearly admission patterns.
- Identify hospitals with higher recorded admission volumes.
- Navigate to detailed patient, operational, financial and clinical analyses.



# Step 4.2: Power BI Dashboard — Page 2: Patient Analytics

## 1. Objective

The Patient Analytics page was designed to analyze patient demographics, admission patterns, medical conditions, clinical test results and patient segmentation.

It provides a more detailed view of patient-related information than the overall healthcare overview page. The objective is to help users explore differences across demographic and clinical categories using interactive visuals and filters.

## 2. KPI Cards

Four KPI cards were created at the top of the dashboard.

| KPI | Displayed value | Purpose |
|---|---|---|
| Abnormal Test % | 33.54% | Shows the percentage of admission records with abnormal test results. |
| Average Patient Age | 51.53 | Shows the average age recorded in the patient dimension. |
| Emergency Admission % | 32.94% | Shows the percentage of admission records classified as Emergency. |
| Long Stay % | 33.36% | Shows the percentage of admission records with a length of stay greater than 20 days. |

These KPIs summarize patient age, clinical test results, admission type and length-of-stay patterns.

## 3. Visualizations

### A. Medical Condition Distribution — Horizontal Bar Chart

**Business question:** How many admission records are associated with each medical condition?

The chart compares admission counts across Arthritis, Diabetes, Hypertension, Obesity, Cancer and Asthma.

It provides a quick comparison of patient admission volume across the recorded medical conditions.

## B. Admissions by Medical Condition and Gender — Clustered Column Chart

**Business question:** How are admission records distributed by medical condition and gender?

The clustered column chart compares female and male admission counts within each medical-condition category.

The legend distinguishes gender, while the medical condition is used as the category axis. This allows users to compare the two recorded gender categories across conditions.

## C. Test Results Distribution — Donut Chart

**Business question:** What proportion of admission records have Normal, Abnormal or Inconclusive test results?

The donut chart shows the distribution of the three test-result categories.

The dataset contains approximately:

- Abnormal: 33.56%
- Normal: 33.36%
- Inconclusive: 33.07%

The chart helps users understand the overall distribution of recorded test results.

## D. Admissions by Medical Condition and Test Result — 100% Stacked Bar Chart

**Business question:** How does the composition of test results vary across medical conditions?

The chart displays the percentage distribution of Abnormal, Inconclusive and Normal test results within each medical-condition category.

Using a 100% stacked bar chart allows comparisons of proportions even when the total admission counts for categories differ slightly.

### E. Clinical Patient Mix by Condition and Test Result — Treemap

**Business question:** How are admission records distributed across combinations of medical condition and test result?

The treemap uses medical condition as the primary category and test result as the secondary grouping. Rectangle sizes represent admission volume, while the segments distinguish the different combinations.

This provides a compact visual overview of the clinical categories in the dataset.

### F. Patient Demographic Analysis — Average Patient Age

The average patient age is presented as a KPI. The Age Group slicer allows users to explore the report by the selected age bands:

- 0–18
- 19–30
- 31–45
- 46–60
- 61–75
- 76+

This supports demographic filtering without repeating an additional age-distribution chart.

## 4. Slicers and Interactive Filters

Four slicers are visible in the report:

1. `medical_condition` — Filters by medical condition.
2. `admission_type` — Filters by Elective, Urgent or Emergency.
3. `gender` — Filters by the recorded gender category.
4. `insurance_provider` — Filters by insurance provider.

The Age Group slicer is also available in the dashboard header.

These slicers allow users to investigate clinical and demographic patterns within selected subsets of the data. Their effects depend on the configured model relationships and visual interactions.

## 5. Patient Segmentation Using a Decomposition Tree

The lower section of the dashboard contains a Decomposition Tree starting with Total Revenue.

The visual allows users to break down recorded revenue by dimensions such as:

- Medical condition
- Admission type
- Insurance provider
- Gender

Users can expand branches to explore how the total revenue is distributed across different categories.

This visual supports interactive drill-down analysis rather than presenting only a fixed chart.

**Interpretation note:** The Decomposition Tree is showing revenue segmentation, not patient counts or causal relationships.

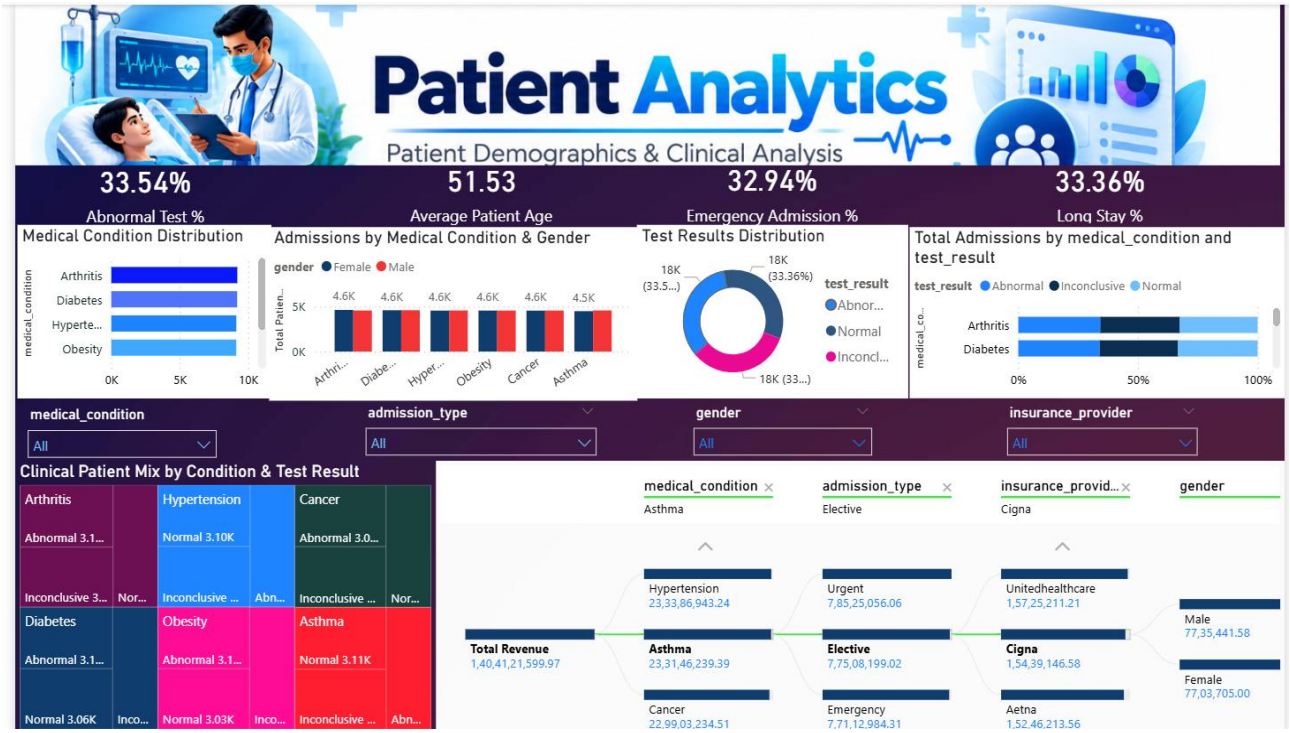
## 6. Business Value

The Patient Analytics page brings together demographic and clinical information to help users:

- Compare admission volumes across medical conditions.
- Examine recorded admission counts by gender.
- Understand the distribution of test results.
- Compare clinical-result proportions across conditions.
- Explore patient segments using age-group and category filters.
- Investigate revenue composition through the Decomposition Tree.

## 7. Page 2 Summary

The Patient Analytics page extends the overall dashboard by focusing on patient demographics, clinical-result patterns and patient-related segmentation. It combines KPI cards, bar charts, a clustered column chart, donut chart, 100% stacked bar chart, treemap, slicers and a Decomposition Tree to support interactive healthcare analysis.




# Step 4.3: Power BI Dashboard — Page 3: Hospital & Operations

## 1. Objective

The Hospital & Operations page was developed to analyze hospital-level admission volumes, average length of stay, recorded revenue and operational indicators.

The purpose of this page is to help users compare hospital performance, identify hospitals with higher recorded revenue, and review admission patterns and longer-stay records.

## 2. KPI Cards

Three KPI cards were created at the top of the dashboard.

| KPI | Displayed value | Purpose |
|---|---|---|

| | | |
|---|---|---|
| Long Stay Admissions | 18.30K | Shows admission records with a length of stay greater than 20 days. |
| Emergency Admissions | 18.07K | Shows admission records classified as Emergency. |
| Average Revenue per Stay Day | 3,399 | Shows the average recorded billing amount per stay day. |

These KPIs provide a quick overview of longer stays, emergency admission volume and the financial measure associated with each stay day.

## 3. Hospital Name Slicer

A `Hospital Name` slicer was added to the top-right section of the page.

It allows users to select a hospital and explore the connected visuals for that hospital, subject to the report's configured interactions and filters.

This supports focused analysis instead of requiring users to examine all hospitals at once.

## 4. Visualizations

### A. Hospital Performance: Admissions vs Average LOS — Combo Chart

**Business question:** How do admission volumes and average length of stay vary across hospitals?

The combination chart uses clustered columns and a line:

- The columns compare admissions by admission type: Elective, Emergency and Urgent.
- The line represents average length of stay.
- Hospital name is used as the category axis.

The chart combines admission volume and stay duration in one view, allowing users to compare two operational measures across the selected hospitals.

### B. Hospital Performance: Admissions vs Revenue — Pie Chart

**Business question:** How is recorded revenue distributed across hospitals?

The screenshot shows a pie chart with hospital names in the legend and revenue values represented by the slices.

It compares each displayed hospital's contribution to the revenue represented in the visual.

**Visual-label note:** The chart title says “Admissions vs Revenue,” but the displayed pie values are revenue-based. For clarity, a more accurate title would be **Revenue Contribution by Hospital**.

## C. Operational Segment Value by Segment — Funnel Chart

**Business question:** How do key admission-volume indicators compare in size?

The funnel chart compares four measures:

- Total Admissions — 54.86K
- Abnormal Test Admissions — 18.40K
- Long Stay Admissions — 18.30K
- Emergency Admissions — 18.07K

A custom `Operational Segments` table and a DAX `SWITCH()` measure were used to supply the values for the funnel visual.

This provides a compact comparison of total admission volume and selected operational categories.

**Interpretation note:** These categories overlap and are not sequential stages in a patient journey. The funnel should therefore be interpreted as a comparison of operational segments, not as a conversion funnel.

## D. Total Revenue by Hospital Name — Treemap

**Business question:** Which hospitals contribute the largest amounts of recorded revenue?

The treemap uses hospital names as categories and total revenue as the value.

The size of each rectangle represents the hospital's recorded revenue contribution. Labels identify the hospitals, allowing users to compare their relative contributions in a compact visual.

The displayed hospitals include Johnson PLC, Smith PLC, Smith Ltd, Group Smith, LLC Johnson and other hospitals.

## 5. Technical Implementation

The page uses the existing MySQL star schema imported into Power BI.

The key measures used for the visuals include:

- Total Admissions
- Emergency Admissions
- Long Stay Admissions
- Average Revenue per Stay Day
- Total Revenue
- Average Length of Stay

A custom operational-segment table was created to organize the funnel categories. A DAX `SWITCH()` measure returns the corresponding KPI for each selected segment.

The hospital slicer and dimension relationships enable hospital-level filtering across connected visuals.

## 6. Business Value

The Hospital & Operations page helps users:

- Compare hospital admission volumes by admission type.
- Review average length of stay across hospitals.
- Explore the distribution of recorded revenue across hospitals.
- Identify hospitals with relatively high revenue contributions.
- Compare total admissions with emergency, abnormal-test and long-stay admission counts.
- Filter the analysis to an individual hospital.

These indicators can support operational review and help prioritize further investigation.

## 7. Limitations

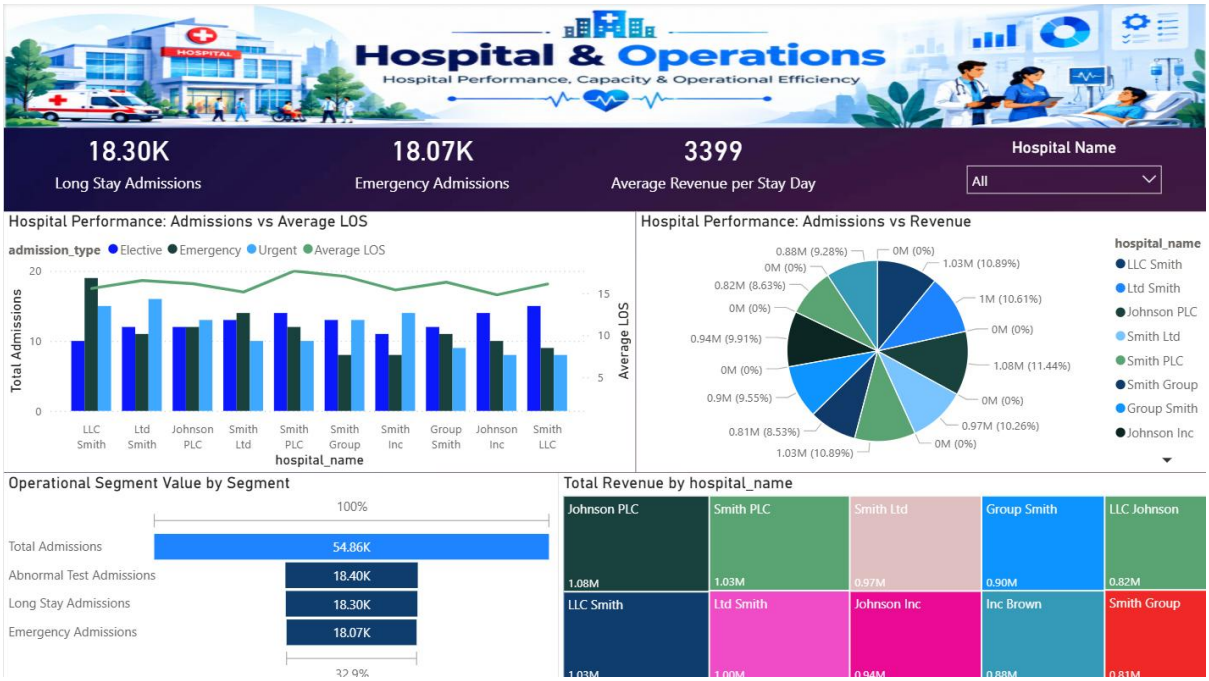
The source dataset does not include hospital bed capacity, actual occupancy, staffing levels or verified operating costs. Therefore, the dashboard cannot independently establish bed shortages, operational inefficiency or profitability.

Recorded revenue represents billing amounts in the dataset, not verified profit or collected cash. The displayed KPIs should be interpreted as analytical indicators rather than definitive measures of hospital quality.

## 8. Page 3 Summary

The Hospital & Operations page combines KPI cards, a combo chart, a revenue pie chart, an operational-segment funnel and a revenue treemap.

Together, these visuals provide an interactive overview of hospital admission patterns, length of stay and recorded revenue contribution within the MediPulse project.



# Step 4.4: Power BI Dashboard — Page 4: Financial Intelligence

## 1. Objective

The Financial Intelligence page was developed to analyze recorded healthcare revenue, billing patterns and financial contributions across medical conditions, admission types, insurance providers and patient age groups.

The objective is to provide a consolidated financial overview and enable users to investigate revenue patterns using interactive filters.

## 2. KPI Cards

Four KPI cards were created at the top of the dashboard.

| KPI | Displayed value | Purpose |
|---|---|---|
| Total Revenue | 1.40bn | Shows the total recorded billing amount. |
| Median Billing | 25,593.88 | Shows the median billing amount, representing the middle value of the billing distribution. |
| Average Billing | 25,594.63 | Shows the average billing amount per admission record. |
| High Billing Admissions | 2,743 | Shows the count of admissions flagged by the project's high-billing review logic. |

The KPI cards provide an overview of financial scale, typical billing values and records identified for further review.

**Interpretation note:** High Billing Admissions represents a review indicator, not confirmed billing errors or fraud.

## 3. Slicers and Filters

Four slicers were placed in the header area.

### A. Date

Allows users to select the admission-date range. The displayed range is 8 May 2019 to 7 May 2024.

### B. Hospital Name

Filters the report by the selected hospital.

### C. Admission Type

Filters the analysis by Elective, Urgent or Emergency admission.

### D. Insurance Provider

Filters the analysis by the recorded insurance provider.

These slicers support focused comparisons across different financial and operational segments.

## 4. Visualizations

### A. Total Revenue and Average Billing by Year and Month — Combo Chart

**Business question:** How do recorded revenue and average billing vary over time?

The combination chart displays:

- Columns for Total Revenue.
- A line for Average Billing.
- A time-based axis organized by year and month.

Using both measures allows users to examine revenue trends alongside changes in the average billing amount.

The visual can help identify periods with different revenue levels and billing patterns. The partial calendar-year coverage at the beginning and end of the dataset should be considered when interpreting yearly patterns.

## B. Revenue by Test Results — Donut Chart

**Business question:** How is recorded revenue distributed across clinical test-result categories?

The donut chart compares the billing contribution associated with:

- Abnormal test results
- Normal test results
- Inconclusive test results

This visual examines the financial distribution across recorded clinical-result categories. It does not imply that test results caused the billing differences.

## C. Total Revenue by Medical Condition — Horizontal Bar Chart

**Business question:** Which medical conditions account for the largest recorded billing amounts?

The chart compares total revenue across Arthritis, Diabetes, Hypertension, Obesity, Cancer and Asthma.

It helps users examine relative billing contributions across the recorded medical-condition categories.

## D. Total Revenue by Admission Type — Donut Chart

**Business question:** How is recorded revenue distributed across admission types?

The donut chart compares total revenue associated with Elective, Urgent and Emergency admissions.

It provides a financial perspective on the admission-type categories, complementing the admission-volume analysis on the overall overview page.

## E. Revenue Contribution by Admission Type — Waterfall Chart

**Business question:** How does the recorded revenue total break down across admission types?

The waterfall chart displays revenue contributions from Elective, Urgent and Emergency admissions, followed by the total.

This provides a cumulative-style view of how the admission-type categories contribute to the overall recorded revenue.

### F. Total Revenue by Age Group — Column Chart

**Business question:** How does recorded revenue vary across patient age groups?

The chart compares total revenue for the following age bands:

- 0–18
- 19–30
- 31–45
- 46–60
- 61–75
- 76+

This helps users explore how revenue is distributed across age groups in the dataset. The visual describes the observed data and does not establish that age causes higher or lower billing.

### G. Total Revenue by Insurance Provider — Horizontal Bar Chart

**Business question:** Which insurance providers account for the largest recorded billing totals?

The chart compares revenue across Cigna, Medicare, Blue Cross, UnitedHealthcare and Aetna.

It allows users to compare the financial contribution associated with each provider and investigate the results further using the Insurance Provider slicer.

## 5. Technical Implementation

The page uses the existing MySQL star schema imported into Power BI.

The principal measures include:

- Total Revenue
- Average Billing
- Median Billing
- High Billing Admissions

Revenue-based visuals aggregate `billing_amount` across the selected categories. Date, hospital, admission-type and insurance-provider slicers provide interactive filtering through the model relationships and configured visual interactions.

The High Billing Admissions KPI uses the project's high-billing review logic. Such records are intended for additional investigation rather than being automatically classified as errors.

## 6. Business Value

The Financial Intelligence page helps users:

- Monitor overall recorded billing revenue.
- Compare average and median billing.
- Explore monthly revenue and average-billing trends.
- Compare revenue by medical condition and admission type.
- Investigate revenue distribution across insurance providers.
- Examine revenue across patient age groups.
- Identify high-billing records for further review.

## 7. Limitations

The source dataset contains billing amounts but does not provide verified collected payments, operating costs, insurance claim settlement status or profit figures.

Therefore, the dashboard measures recorded billing revenue rather than profit, cash collection or confirmed financial performance.

Differences between categories should be interpreted descriptively. High-billing flags are review indicators and do not independently establish overbilling or fraud.

## 8. Page 4 Summary

The Financial Intelligence page combines financial KPIs, date and category slicers, a revenue trend combo chart, donut charts, horizontal bar charts, a waterfall chart and an age-group column chart.

Together, these visuals provide an interactive view of recorded billing patterns and revenue contributions across healthcare categories.




# Step 4.5: Power BI Dashboard — Page 5: Patient Outcomes & Clinical Insights

## 1. Objective

The Patient Outcomes & Clinical Insights page was developed to analyze clinical test results, medical conditions, medication patterns and patient length-of-stay trends.

The dashboard combines clinical KPIs, medication-level comparisons, medical-condition distributions and interactive filters to help users explore patient admission records from different clinical perspectives.

## 2. KPI Cards

Four KPI cards were created at the top of the dashboard.

| KPI | Displayed value | Purpose |
|---|---|---|
| Abnormal Test Admissions | 18,399 | Shows admission records classified with abnormal test results. |
| Inconclusive Test Admissions | 18,159 | Shows admission records classified with inconclusive test results. |
| Normal Test Admissions | 18,302 | Shows admission records classified with normal test results. |
| Avg Age – Abnormal Results | 51.63 | Shows the average age associated with records having abnormal test results. |

These KPIs summarize the distribution of recorded test-result categories and the average age associated with abnormal-result records.

## 3. Slicers and Filters

Four slicers were added to the top-right section of the dashboard.

### A. Test Result

Filters admission records by Abnormal, Normal or Inconclusive test results.

### B. Admission Type

Filters records by Elective, Urgent or Emergency admission.

### C. Medication Name

Allows users to investigate records associated with a selected medication.

### D. Medical Condition

Filters the dashboard by the recorded medical condition.

These slicers support interactive exploration of clinical and medication-related patterns.

# 4. Visualizations

## A. Medical Condition, Medication and Total — Matrix

**Business question:** How are admission records distributed across medical conditions and medications?

The matrix displays medical conditions as rows and medication names as columns. Each cell shows the corresponding admission count, with row and column totals.

It enables users to compare medication-related admission counts across Arthritis, Asthma, Cancer, Diabetes, Hypertension and Obesity.

## B. Total Admissions by Medication Name and Test Result — Waterfall Chart

**Business question:** How do admission counts vary across medication and test-result categories?

The waterfall chart displays changes in admission counts across medication and test-result combinations.

It provides a detailed view of category-level differences. Because the visual contains positive and negative changes, its interpretation depends on the order and calculation used to construct the waterfall.

## C. Average LOS by Year — Line and Area Chart

**Business question:** How has the average length of stay changed over time?

The chart plots average length of stay by year, showing how the measure varies across the available admission years.

The displayed trend includes a decline through 2022, followed by an increase in later years. This can help users identify periods that may warrant further investigation.

## D. Patient Volume by Medical Condition — Treemap

**Business question:** Which medical conditions account for the largest admission volumes?

The treemap compares admission counts across the six medical conditions. The rectangle sizes represent patient admission volume, while the labels identify the medical-condition categories.

It provides a compact comparison of the clinical mix represented in the dataset.

## E. Total Admissions by Medication Name and Test Result — 100% Stacked Column Chart

**Business question:** What is the percentage distribution of test results within each medication category?

The chart compares Abnormal, Inconclusive and Normal test-result proportions across medication categories.

The 100% stacked format emphasizes the relative composition of each medication category rather than only the total count.

## F. Billing Amount vs Length of Stay — Donut Chart

**Business question:** How is the displayed billing measure distributed across medical conditions?

The screenshot shows a donut chart with medical condition in the legend and billing-related values in the visual.

This provides a category-based financial view within the clinical analysis page.

**Important:** The visual title says “Billing Amount vs Length of Stay,” but the displayed donut appears to use billing amount as its value and medical condition as its category. The title and field configuration should be checked so that the visual accurately describes what it measures.

## G. Total Admissions by Medical Condition and Test Result — Matrix-Style Visual

**Business question:** How do admission counts and test-result categories compare across medical conditions?

The visual presents medical conditions alongside the counts for Abnormal, Inconclusive and Normal test results.

It allows users to compare the recorded clinical-result distribution within each condition and investigate categories with relatively different counts.

## 5. Technical Implementation

The page uses the existing MySQL star schema imported into Power BI.

The visuals rely on measures such as Total Admissions, Average Length of Stay and billing-related measures, combined with the medical-condition, medication, test-result and admission-type dimensions.

The slicers provide interactive filtering through the configured data model relationships and visual interactions.

## 6. Business Value

The Patient Outcomes & Clinical Insights page helps users:

- Compare the distribution of normal, abnormal and inconclusive test results.
- Explore admission volume across medical conditions.
- Compare medication and test-result combinations.
- Review yearly average length-of-stay trends.
- Examine clinical-result proportions across medication categories.
- Investigate billing-related patterns across medical conditions.
- Filter the analysis by test result, admission type, medication and medical condition.

## 7. Limitations

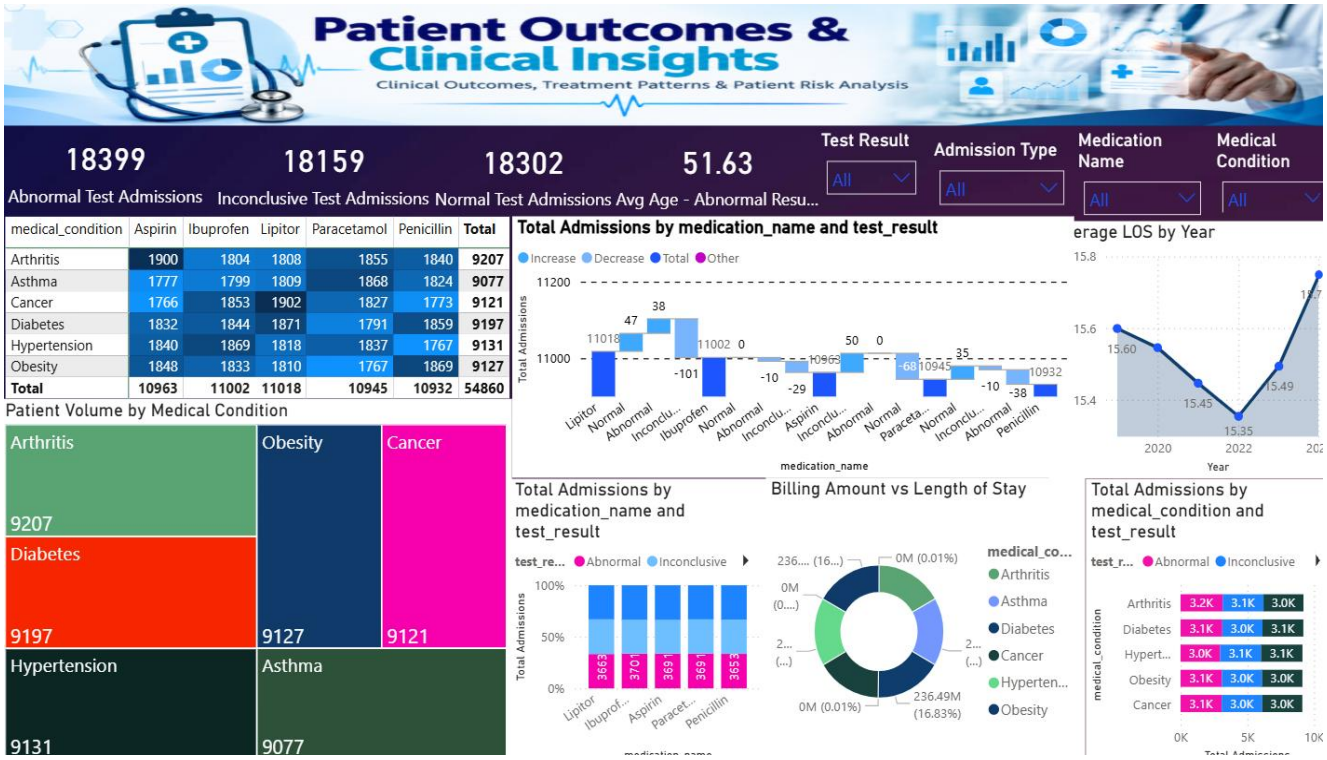
The dataset contains admission records, medical conditions, medications, billing amounts and test-result categories. It does not contain detailed treatment outcomes, recovery status, readmission history, mortality data or verified clinical effectiveness measures.

Therefore, the dashboard supports analysis of recorded clinical categories and admission patterns. It cannot independently establish treatment effectiveness, patient recovery or causal relationships between medication and outcomes.

## 8. Page 5 Summary

The Patient Outcomes & Clinical Insights page combines clinical KPI cards, a matrix, a waterfall chart, a yearly trend chart, a treemap, a 100% stacked column chart and a billing-related donut chart.

Together, these visuals provide an interactive view of clinical-result distributions, medication patterns, medical conditions, admission volume and length-of-stay trends.




# Step 5: Challenges Faced and Solutions Implemented

During the development of the MediPulse project, several challenges were encountered during data analysis, data cleaning, MySQL integration, data modeling and Power BI

dashboard development. These challenges were resolved through systematic validation, debugging and iterative improvements.

## 5.1 Python Data Cleaning Challenges

### Challenge 1: Duplicate Records

**Problem:** The original healthcare dataset contained 534 exact duplicate rows. These duplicates could inflate admission counts and financial calculations.

**Solution:** Python Pandas was used to identify and remove exact duplicate records. Duplicate records were checked before and after cleaning to confirm that the cleaned dataset contained no exact duplicate rows.

**Result:** The analytical dataset was reduced from 55,500 to 54,860 records after duplicate removal and other cleaning steps.

### Challenge 2: Negative Billing Amounts

**Problem:** The dataset contained 108 records with negative billing amounts. Including these records without understanding their meaning could distort revenue and average-billing calculations.

**Solution:** The negative-billing records were exported to a separate review CSV file. They were excluded from the main analytical dataset because their validity could not be established from the available columns alone.

**Result:** The cleaned analytical dataset contained no negative billing amounts. The excluded records were preserved separately for review.

### Challenge 3: Data Quality Validation

**Problem:** Before loading the dataset into MySQL, it was necessary to verify missing values, data types, date order, age ranges and length-of-stay calculations.

**Solution:** Pandas was used to inspect the dataset, validate date and numeric fields, recalculate Length of Stay and check for remaining data-quality issues.

**Result:** The final cleaned dataset contained 54,860 records and 36 columns, with no reported missing values, exact duplicate rows, negative billing amounts or invalid length-of-stay records.

## 5.2 MySQL Import Challenges

### Challenge 4: Slow CSV Import Through MySQL Workbench

**Problem:** Importing the cleaned CSV through the MySQL Table Data Import Wizard was slow and did not provide a reliable, efficient workflow.

**Solution:** The staging table was prepared, and the CSV was loaded using the `LOAD DATA LOCAL INFILE` statement after enabling local file loading.

**Result:** All 54,860 cleaned records were successfully imported into the `staging_admissions` table.

### Challenge 5: LOCAL INFILE Error 2068

**Problem:** The initial `LOAD DATA LOCAL INFILE` command failed because the MySQL client connection restricted local file loading.

**Solution:** The server's `local_infile` setting was enabled, and `OPT_LOCAL_INFILE=1` was added to the MySQL Workbench connection's advanced settings. The connection was then re-established before retrying the import.

**Result:** The local CSV import succeeded, and the staging-table record count was verified.

## 5.3 Data Modeling Challenges

### Challenge 6: Designing a Star Schema

**Problem:** The raw healthcare dataset was stored in a flat table. A structured model was needed to support efficient analysis across patients, hospitals, conditions, insurance providers, medications, admission types, test results and dates.

**Solution:** A star schema was created in MySQL with a central `fact_admission` table and dimension tables, including `dim_patient`, `dim_hospital`, `dim_date`, `dim_medical_condition`, `dim_insurance`, `dim_medication`, `dim_admission_type` and `dim_test_result`.

Primary keys, foreign keys and indexes were configured where appropriate. Fact-table joins were validated before loading the records.

**Result:** The star schema contained 54,860 fact records and eight dimension tables, with the intended one-to-many relationships from the dimensions to the fact table.

**Design consideration:** The source dataset did not provide a reliable patient identifier. Therefore, the patient dimension was constructed using available demographic attributes. Repeated names or similar demographic details should not automatically be interpreted as proof of the same real-world patient.

## 5.4 Power BI and MySQL Connectivity Challenges

### Challenge 7: MySQL Connector Compatibility

**Problem:** Power BI initially failed to connect to MySQL because the required MySQL Connector/NET provider was not being recognized correctly. Multiple provider registrations in the .NET configuration also complicated troubleshooting.

**Solution:** The Connector/NET installation and provider configuration were investigated. Duplicate provider registrations were removed carefully, and the working configuration was restored and verified.

**Result:** The MySQL connector became available in Power BI Desktop, allowing the project database to be accessed.

### Challenge 8: Importing the Correct Tables into Power BI

**Problem:** The database contained both analytical dimension/fact tables and the original staging table. Loading the staging table unnecessarily could introduce redundant data into the report model.

**Solution:** The eight dimension tables and the fact table were selected in Power Query, while `staging_admissions` was excluded from the report model.

**Result:** Nine analytical tables were loaded into Power BI for reporting.

### Challenge 9: Verifying Model Relationships

**Problem:** Dashboard measures and slicers depend on correct relationships between fact and dimension tables.

**Solution:** The Power BI Model view was inspected to verify the intended one-to-many relationships between the dimension tables and `fact_admission`.

**Result:** The star-schema relationships were established for the report's interactive filtering and aggregation.

## 5.5 DAX and Dashboard Development Challenges

### Challenge 10: Creating Measures and Calculated Columns

**Problem:** Some DAX expressions returned errors because a column was referenced where a single scalar value was required, or because a name was already used by an existing measure.

**Solution:** Expressions were reviewed to distinguish measures from calculated columns. Appropriate aggregation and row-level logic were used, and conflicting names were changed when necessary.

**Result:** Measures and calculated columns were created for admission counts, revenue, average billing, average length of stay, age groups, emergency admissions, long stays and other analytical indicators.

### Challenge 11: Comparing Categories Without Misleading Interpretations

**Problem:** Several visualizations compared admission counts, revenue, test-result distributions and operational categories. Some charts could be misinterpreted if their titles or visual structures did not match the underlying measures.

**Solution:** Visuals were selected according to the business question, including bar charts, clustered columns, combo charts, donut charts, treemaps, a waterfall chart and a decomposition tree. Ambiguous chart titles and overlapping operational categories were identified for review.

**Result:** The report was organized into five pages: Overview, Patient Analytics, Hospital & Operations, Financial Intelligence, and Patient Outcomes & Clinical Insights.

## 5.6 Summary of Key Challenges

| Area | Challenge | Resolution |
|---|---|---|
| Python | Duplicate records | Identified and removed exact duplicates |
| Python | Negative billing | Exported for review and excluded from the main analytical dataset |
| Python | Data quality | Validated fields, dates and length of stay |

| | | |
|---|---|---|
| MySQL | Slow CSV import | Used `LOAD DATA LOCAL INFILE` |
| MySQL | Error 2068 | Corrected client/server local-file loading configuration |
| Data Modeling | Flat-table structure | Created a star schema |
| Power BI | Connector failure | Resolved Connector/NET provider configuration |
| Power BI | Redundant staging data | Excluded staging table from the report model |
| Power BI | Relationship validation | Verified dimension-to-fact relationships |
| DAX | Formula and naming errors | Corrected expressions and naming conflicts |
| Dashboard Design | Potentially misleading visuals | Reviewed chart selection, titles and interpretation |

## 5.7 Conclusion

The challenges encountered during MediPulse helped establish a more reliable analytical workflow, from Python-based data cleaning to MySQL storage, dimensional modeling and Power BI reporting.

The project demonstrates practical troubleshooting across data preparation, database integration, analytical modeling and dashboard development. It also highlights the importance of validating source data and avoiding conclusions that cannot be supported by the available dataset.

# Data Modeling and Relationship Design in Power BI

### Objective

After importing the analytical tables from MySQL into Power BI, a relational data model was configured to support interactive analysis and consistent reporting.

## Star Schema Structure

The data model uses `fact_admission` as the central fact table. It stores admission-level measures and foreign keys that connect each record to the relevant dimension tables.

The dimension tables provide descriptive information for analyzing admissions from different perspectives.

**Central Fact Table**

- `fact_admission` — Contains admission records, billing amounts, length of stay and foreign keys.

**Dimension Tables**

1. `dim_patient` — Patient demographic attributes, including age, gender and blood type.
2. `dim_hospital` — Hospital information.
3. `dim_date` — Calendar attributes such as date, month, quarter and year.
4. `dim_medical_condition` — Medical-condition categories.
5. `dim_insurance` — Insurance provider information.
6. `dim_medication` — Medication categories.
7. `dim_admission_type` — Elective, Urgent and Emergency admission categories.
8. `dim_test_result` — Normal, Abnormal and Inconclusive test-result categories.

## Relationships

One-to-many relationships were configured between the dimension tables and `fact_admission`. The dimension tables provide descriptive categories, while the fact table contains the records and numerical measures used in calculations.

These relationships allow Power BI slicers and visualizations to filter and aggregate admission data across dimensions.

## Why This Model Was Used

- Reduces the need to analyze every category from a single flat table.
- Supports consistent aggregation of admission and billing measures.
- Enables interactive filtering by hospital, date, patient attributes, medical condition, insurance, medication, admission type and test result.

- Provides a structured foundation for DAX measures and dashboard visualizations.

## Validation

The model relationships were inspected in Power BI Model view to verify the intended connections between the dimension tables and the central fact table.

